Snoring: a Noise in Defect Prediction Datasets

Agenda

- Introduction
  - Context / Definitions / Aim
- RQ1: To what extent do defects sleep?
  - Rationale / Design / Results / Discussion
- RQ2: To what extent do classes snore?
- RQ3: To what extent does snoring impact the evaluation of classifiers accuracy?
- RQ4: To what extent does snoring impact defect prediction accuracy?
- RQ5: To what extent is no data better than snoring data?
- Conclusions
- Future works

Introduction - context

- Defect prediction aims at identifying software artifacts that are likely to exhibit a defect.
- The main purpose of defect prediction is to reduce the cost of testing and code review, by letting developers focus on specific artifacts.
- Ultimately, the reliability of a prediction model depends on the quality of the dataset.
  - Previous works have identified sources of noise in datasets, including defect misclassification and defect origin, and proposed solutions to deal with them.

Within-project defect prediction

- Our context is the classification of defective classes in a specific release.

Sleeping Defect (Dormant Bug)

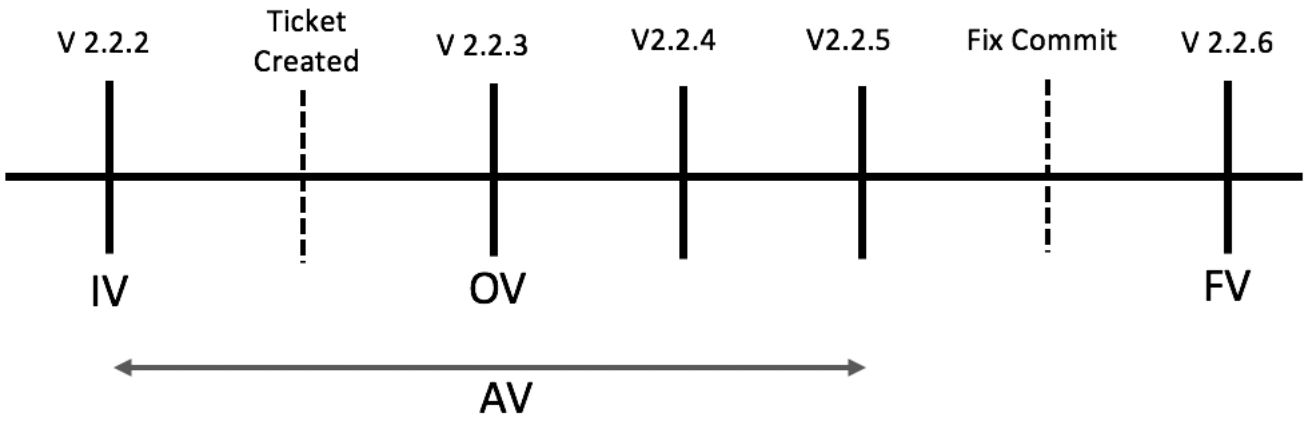
- It is possible that a defect is only discovered or fixed several releases after its introduction
  - Post Release Defects

An example of project where events (I = Injected, N = noting, and F = fixed) happen in four releases and three different classes.

	r1	r2	r3
C1	IF	I	F
C2	I	N	F
C3	II	F	F

Snoring classes

- The intuition of the paper is that **the existence of a defect cannot be known before the defect is fixed**.



Bailey et al. “Can We Rely on Affect Version when Creating Software Defectiveness Datasets?”

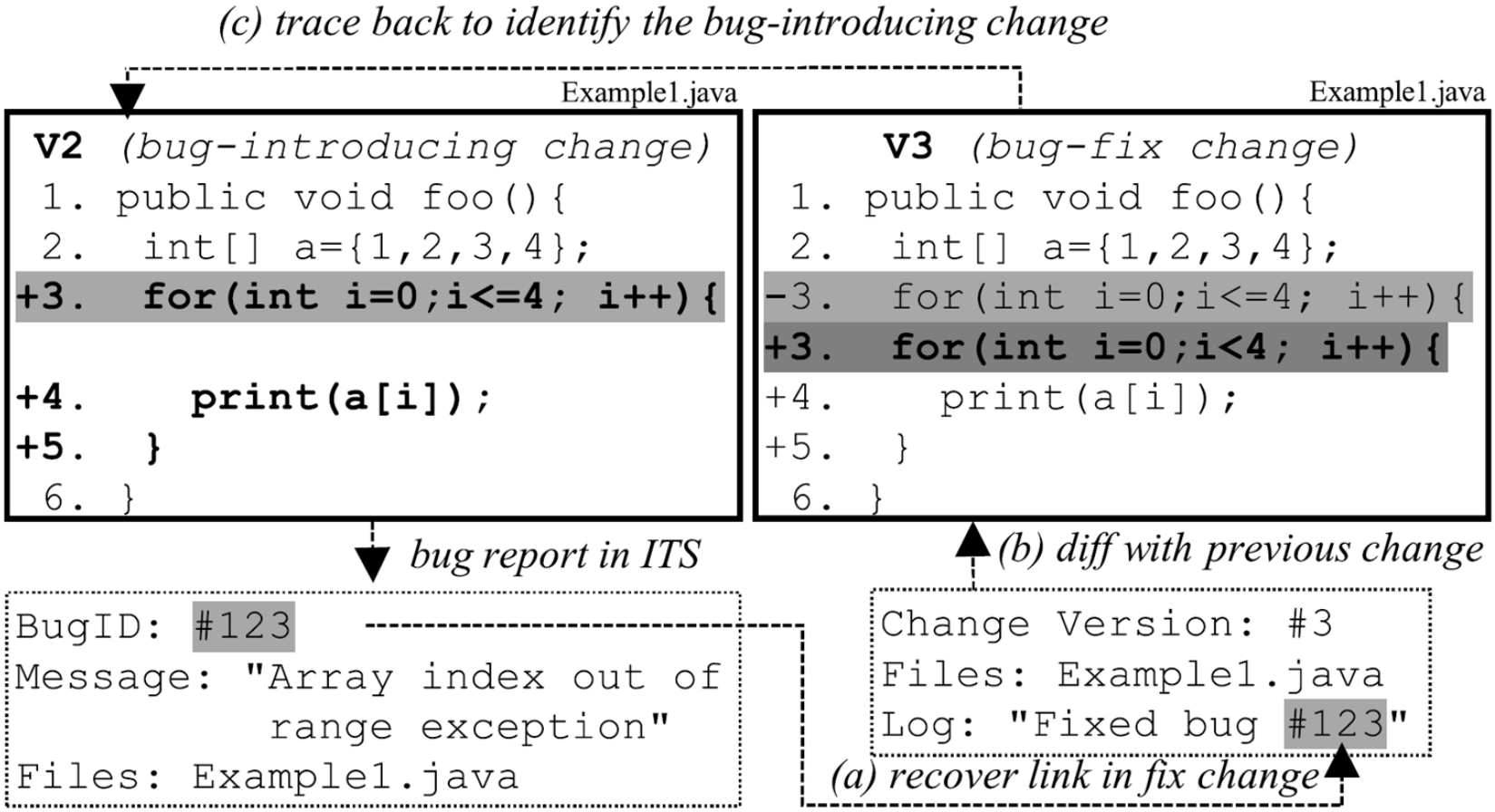
How defects are assigned to classes

- Consider the earliest “Affect Version” in JIRA
- Otherwise SZZ

INTRODUCTION-SZZ

- SZZ : Process to identify when a defect has been actually introduced in a software project
  - Exploits the versioning system annotation mechanism (e.g. git blame)
- Determine, for the source code lines that have been changed in a defect fix, when they have last been changed before such a fix
- Limitations
  - Code additions
  - Merely 26–29% of BICs can be foundusing the algorithm based on the assumption that “a given bugwas introduced by the lines of code that were modified to fix it”.

EXAMPLE OF SZZ



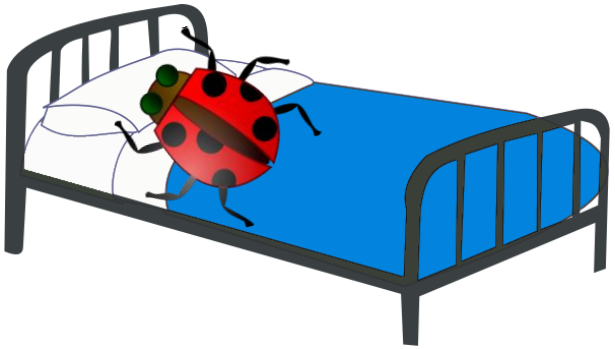
"The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study"

Introduction-Snoring

- Datasets are biased (i.e., many FN) because the measurement upon which the datasets are computed (e.g., SZZ), cannot consider the bugs that *exist and are not yet fixed*.
  - This might introduce a bias in the dataset, i.e., treating the intermediate releases as defect-free and the latter as defect-prone.

INTRODUCTION-SNORING

- We call "**snoring classes**" the phenomenon in which classes are affected only by defects that are not yet fixed, i.e., FN.
  - Contain sleeping defect
  - Produce noise in defect prediction datasets



hospital

100x7 dataset

	1 LastName	2 Sex	3 Age	4 Weight	5 Smoker	6 BloodPressure	7 Trials
1	YPL-320	'SMITH'	Male	38	176	93	18
2	GLI-532	'JOHNSO			163	77	[11,13,22]
3	PNI-258	'WILLIAM			131	83	[]
4	MIJ-579	'JONES'			133	75	[6,12]
5	XLK-030	'BROWN			119	80	[14,23]
6	TFP-518	'DAVIS'			142	70	19
7	LPD-746	'MILLER'			142	88	13
8	ATA-945	'WILSON			180	82	[]
9	VNL-702	'MOORE'			183	78	2
10	LQW-768	'TAYLOR'	Female	31	132	0	118	86	11

AN EXAMPLE OF PROJECT WHERE EVENTS (I = INJECTED, N = NOTING, AND F = FIXED) HAPPEN IN FOUR RELEASES AND THREE DIFFERENT CLASSES.

	r1	r2	r3
C1	IF	I	F
C2	I	N	F
C3	II	F	F

THE POST-RELEASE DEFECTIVENESS OF A CLASS IN A SPECIFIC RELEASE AS COMPUTED AT *r2*.

Classes	r1
C1	ND
C2	ND
C3	D

THE POST-RELEASE DEFECTIVENESS OF A CLASS IN A SPECIFIC RELEASE AS COMPUTED AT *r3*.

Classes	r1	r2
C1	ND	D
C2	D	D
C3	D	D

AN EXAMPLE OF PROJECT WHERE EVENTS (I = INJECTED, N = NOTING, AND F = FIXED) HAPPEN IN FOUR RELEASES AND THREE DIFFERENT CLASSES.

	r1	r2	r3
C1	IF	I	F
C2	I	N	F
C3	II	F	F

Table 1.4: The accuracy of the dataset maker performed in *r2*.

Classes	r1
C1	TN
C2	FN
C3	P

Table 1.5: The accuracy of the dataset maker performed in *r3*.

Classes	r1	r2
C1	TN	P
C2	P	P
C3	P	P

Introduction-Questions

1. To what extent do defects sleep?
2. To what extent do classes snore?
3. To what extent does snoring impact the evaluation of classifier accuracy?
4. To what extent does snoring impact defect prediction accuracy?
5. To what extent is no data better than snoring data?

RQ1: To what extent do defects sleep? - Rationale

- We want to know how many releases elapse between injection and fix of defects
  - Sleeping defects unknown to the dataset maker

RQ1: To what extent do defects sleep? – Design : Project Selection

- 20 large Apache projects
- Issue tracking in Jira
- Source code versioned in Git
- At least 10 releases
- Most commits related to Java code
- Highest proportion of bug-code linkage

Table 3.1: Details of the used projects.

Project	Releases	Days	Commits	Defects
AVRO	46	3528	1770	114
CHUKWA	11	3638	849	7
FALCON	31	2450	2227	219
GIRAPH	10	2719	1096	123
IVY	17	4977	2973	133
OPENJPA	31	4671	4978	380
PROTON	51	2513	3929	53
SSHD	26	3652	1589	124
STORM	36	2634	9754	442
THRIFT	12	3926	5627	72
WHIRR	8	1788	569	20
ZEPPELIN	14	2067	4048	200
ZOOKEEPER	49	3923	1820	219
BOOKKEEPER	22	2880	2056	184
CONNECTORS	118	3311	4672	261
CRUNCH	17	2663	1055	132
SYNCOPE	48	3214	6320	296
TAJO	13	2407	2273	286
TEZ	34	2163	2661	559
TOMEE	22	4792	12135	277

RQ1: To what extent do defects sleep? - Design

- Dependent variable: # of releases a bug slept
  - Measured as # of releases between when the bug injected to when the bug fixed
- Independent variable: Name of the bug in JIRA

RQ1: To what extent do defects sleep? - Results

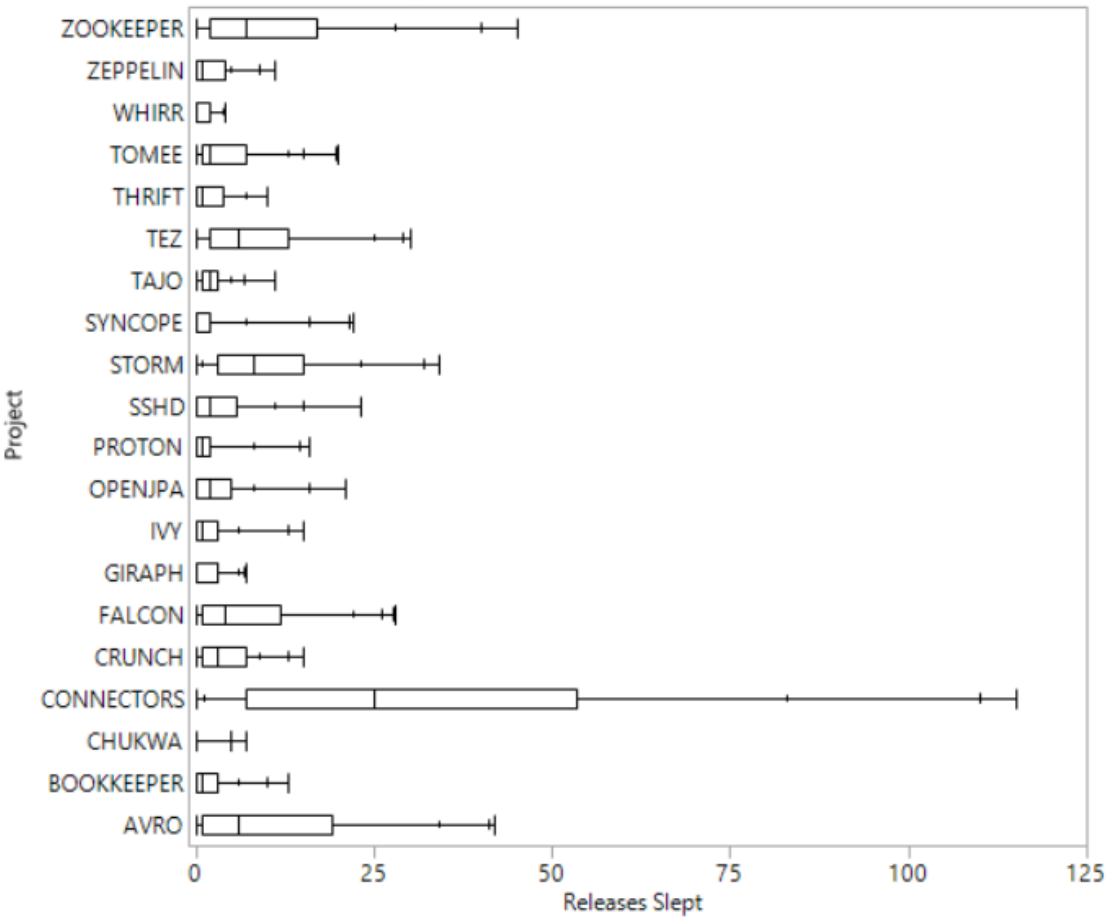


Figure 4.1: Distribution of the number of releases slept by post-release defects.

RQ1: To what extent do defects sleep? - Results

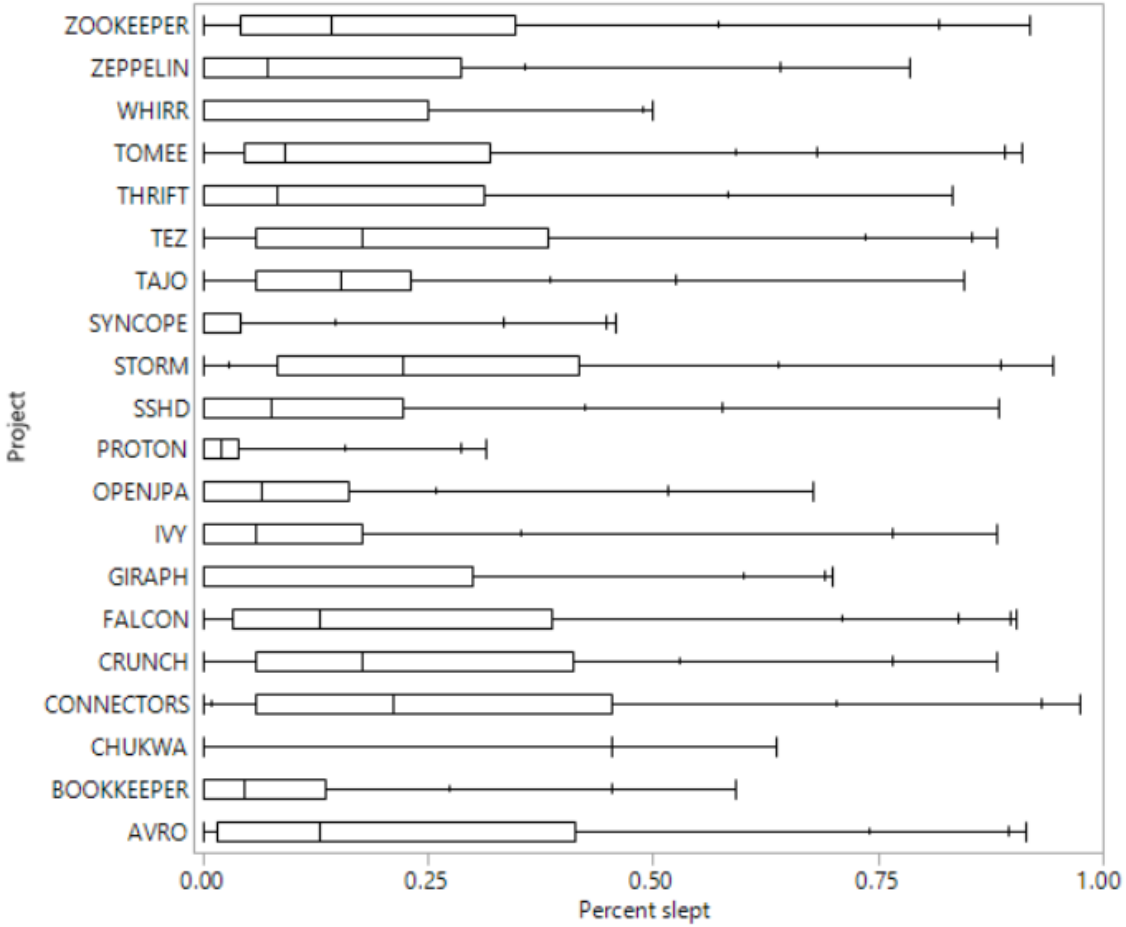


Figure 4.2: Distribution of the number of releases slept by post-release defects.

RQ1: To what extent do defects sleep? - Discussion

- The mean bug sleeps for 7 releases.
- Most of the defects in a project slept for more than 19% of the existing releases
- The average bug sleeps a number of releases that varies in the range [4%, 41%], among projects.
- Extremes
  - LUCENE-3672 : 84 releases, change in imported libraries
  - STRATOS-1653: 51/53 releases, wrong method call

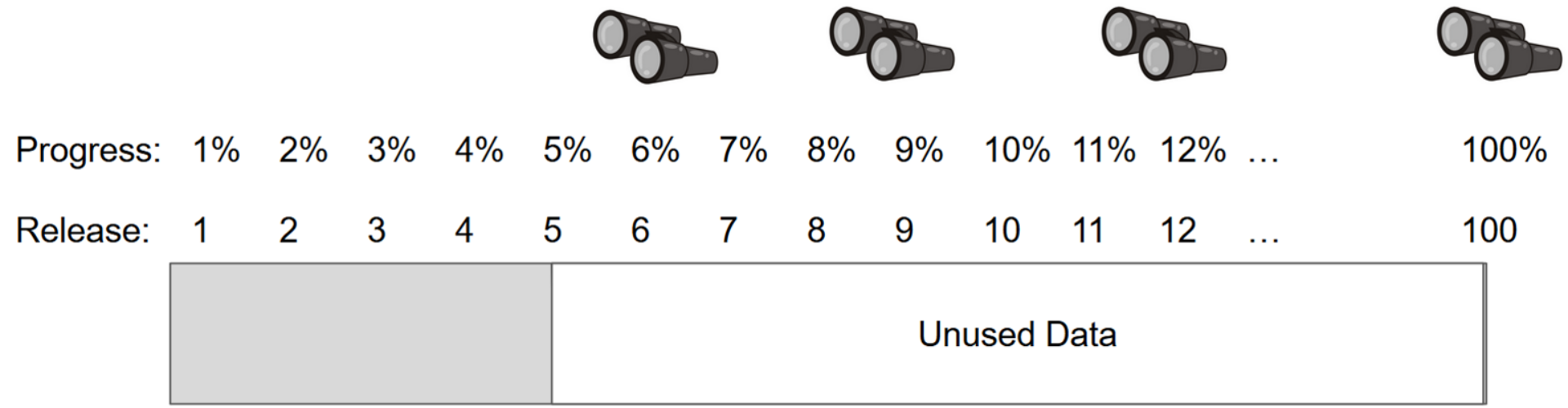
RQ2: To what extent do classes snore? - Rationale

- A sleeping defect does not always produce one snoring class
  - 1) **Another non-snoring defect exists – No noise**
  - 2) Single defect can impact multiple class – Noise in more than one class
- We are interested in measuring to what extent classes snore

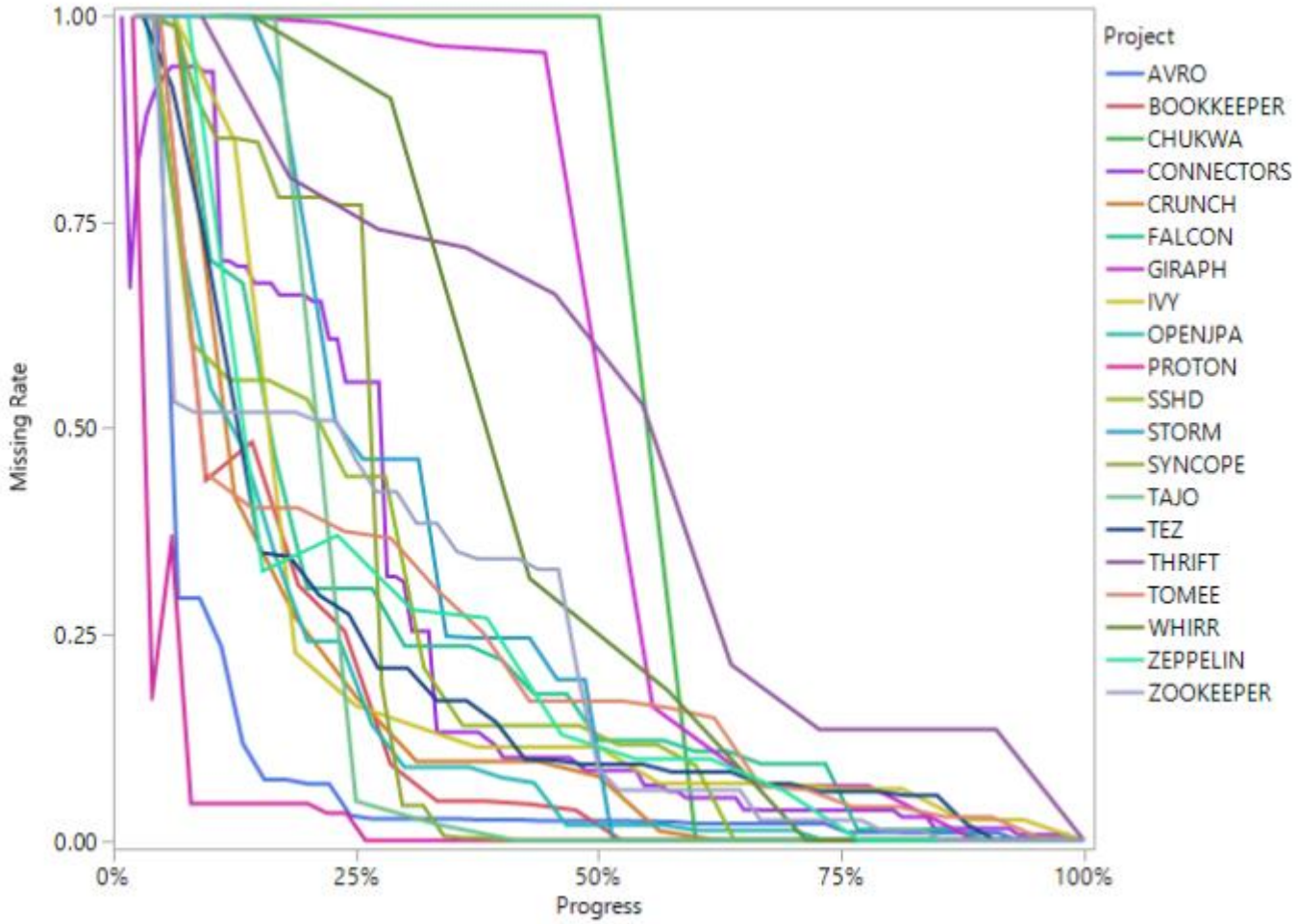
RQ2: To what extent do classes snore? - Design

- Dependent variable: Missing rate
  - False negative rate, Type I error or 1 – Recall
  - the proportion of defective classes that are not identified as defective.
  - FN/(FN+TP).
- Independent variable: Progress
  - We measure how the missing rate of the **5%** sub-dataset varies over time (i.e., throughout different releases).
  - Progress = measuring release / # releases
    - For instance, progress is 100% when measuring the sub-dataset from the last release of the project.

RQ2: To what extent do classes snore? - Design



RQ2: To what extent do classes snore? - Results



RQ2: To what extent do classes snore? - Results

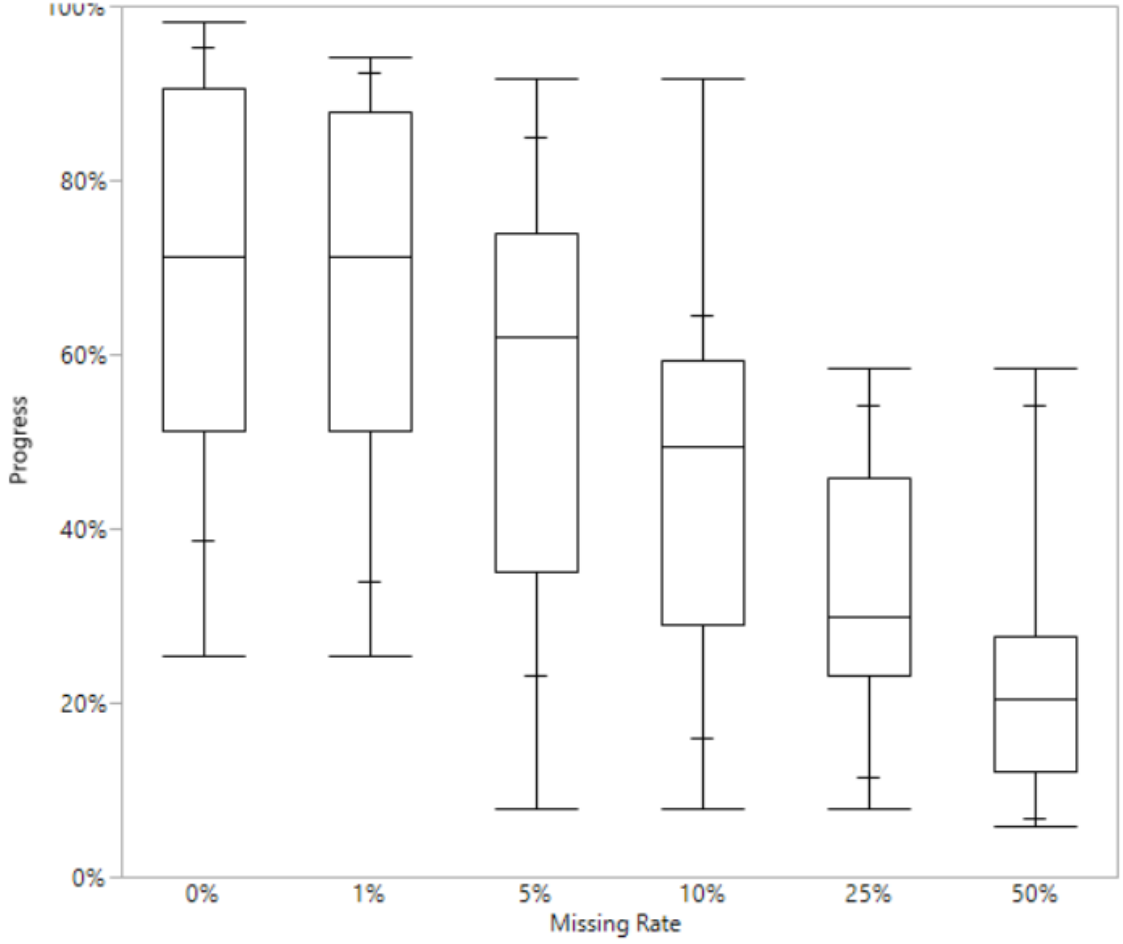


Figure 4.4: Distribution, among datasets, of percentage of releases (Progress) that needs to be removed to achieve a specific missing rate.

RQ2: To what extent do classes snore? - Results

Table 4.1: Median among datasets of the minimum percentage of releases (Progress) that needs to be removed to achieve a specific missing rate.

Missing Rate	Progress
0	0.71
0.01	0.71
0.05	0.62
0.1	0.49
0.25	0.3
0.5	0.2

RQ2: To what extent do classes snore? - Discussion

- For the majority of the projects:
  1. the missing rate is more than 50% unless we remove more than 20% of the releases
  2. the missing rate is more than 5% unless we remove more than 62% of the releases
  3. the missing rate is not null unless we remove more than 71% of the releases
  4. the missing rate is more than 25% even if we remove 30% of releases.

RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? - Rationale

- Researchers evaluate classifiers and related technologies (feature selection, tuning).
- Snoring might impact classifier accuracy Biases classifier evaluations
- Are evaluation studies impacted by snoring data?

RQ3: Design

- Dependent variables:
  - TP/FN/TN/FP
  - Precision/Recall
  - F1
  - Cohen's Kappa
  - AUC
  - Matthews
  - For each of these we computed
    - Relative Bias: relative distance between accuracy of a classifier trained on noisy, noise-free data
    - Selection Accuracy: Best classifiers matched for a specific metric, and project with, without snoring
    - Ranking Accuracy
- Independent variable: Presence or absence of snoring noise in both train and test sets

RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? – Design : Classifiers

- Decision Stump
- Decision Table
- IBK
- J48
- Jrip
- KStar
- Naïve Bayes
- Naïve Bayes Updateable
- OneR
- PART
- Random Forest
- REPTree

RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? – Design

- 66/33 % ordered holdout (train, test split)
- To avoid that noise impacts our own measurement we remove releases from the end of our measurement according to results from RQ2, target 1% snoring in noise-free dataset
  - E.g. 18/36 releases removed from Apache Storm

RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? - Design

- The same entities are used for Tr and Te, the difference is only on the point of time when the class defectiveness is observed.







RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? – Design : Features

Table 3.2: Defect prediction features.

Metric	Description
Size	Lines of code(LOC)
LOC Touched	Sum over revisions of LOC added + deleted
NR	Number of revisions
Nfix	Number of bug fixes
Nauth	Number of authors
LOC Added	Sum over revisions of LOC added + deleted
MAX LOC Added	Maximum over revisions of LOC added
AVG LOC Added	Average LOC added per revision
Churn	Sum over revisions of added deleted LOC
Max Churn	Maximum churn over revisions
Average Churn	Average churn over revisions
Change Set Size	Number of files committed together
Max Change Set	Maximum change set size over revisions
Average Change Set	Average change set size over revisions
Age	Age of Release
Weighted Age	Age of Release weighted by LOC touched

RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? - Results

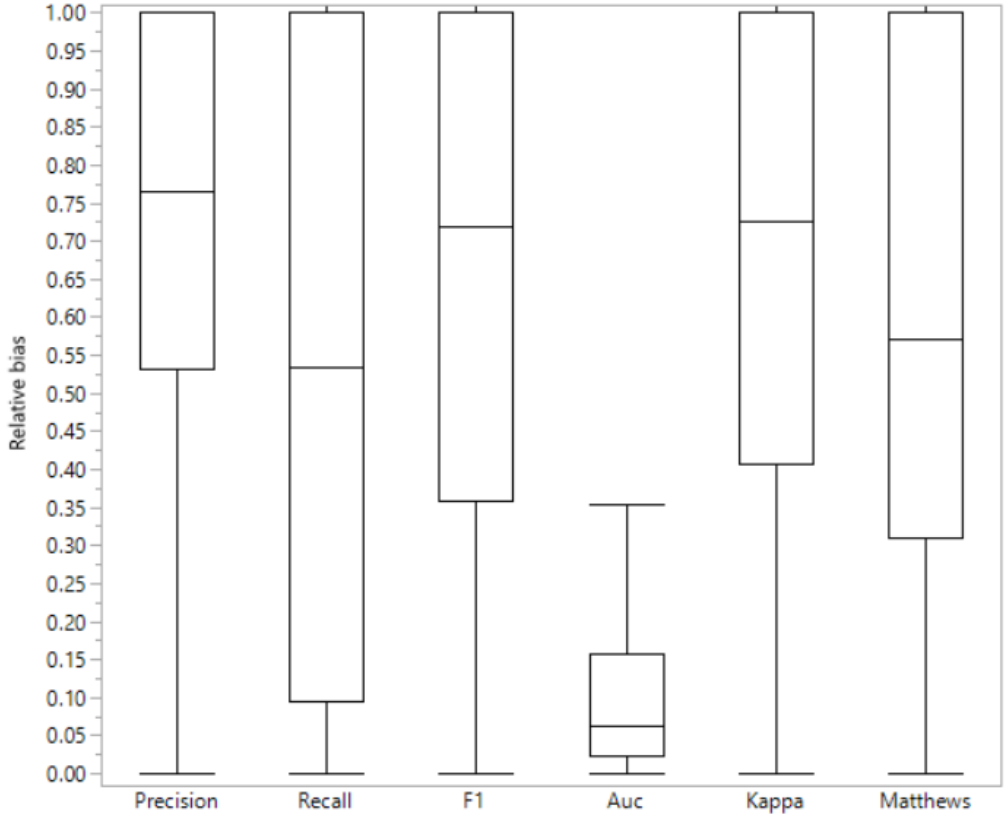


Figure 4.6: Distribution of relative bias, among classifiers and datasets.

RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? - Results

Table 4.3: Average relative bias (%), among classifiers and datasets, in specific metrics.

Precision	Recall	F1	Auc	Kappa	Matthews
0.98	1.11	0.96	0.11	1.21	0.92

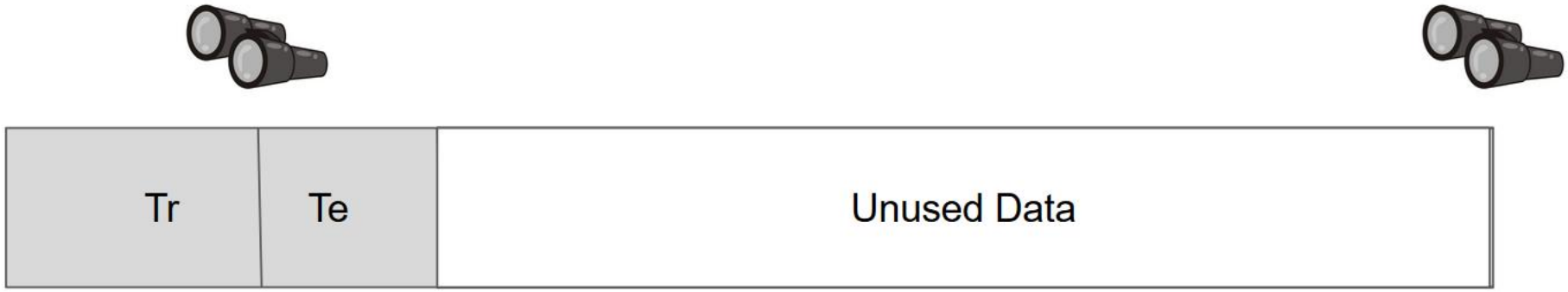
RQ3: To what extent does snoring impact the evaluation of classifiers accuracy? - Discussion

- With snoring data, we only picked the best classifier 45% (49/108 cases)
  - CHUKWA, best classifier always picked
  - OPENJPA, TAJO, ZEPPELIN, best classifier never picked
- Relative bias high for all metrics other than AUC
- Correlation strong????

RQ4: To what extent does snoring impact defect prediction accuracy? - Rationale

- Snoring is a type of noise in defect prediction datasets
- Noise impacts prediction models
- We want to know the extent

RQ4: To what extent does snoring impact defect prediction accuracy? - Design



RQ4: To what extent does snoring impact defect prediction accuracy? - Design

- Dependent Variable and Independent Variable same as RQ3
- TrNS,TeNS and TrS,TeS
  - NO TeS
  - We test the accuracy of a model trained on snoring data against a test set without snoring

RQ4: To what extent does snoring impact defect prediction accuracy? - Results

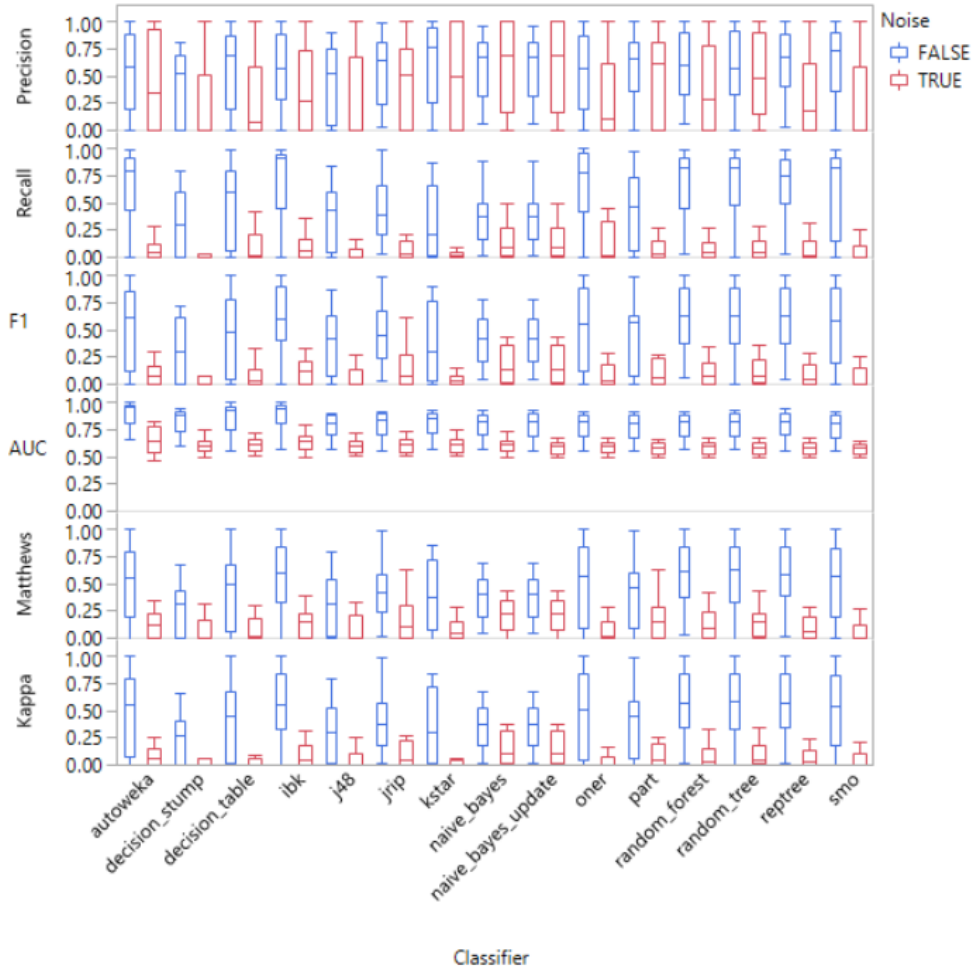


Figure 4.7: Distribution of accuracy, among datasets, of different classifiers (x-axis), in the cases of with, or without, the snoring noise.

RQ4: To what extent does snoring impact defect prediction accuracy? - Results

Table 4.7: Statistical test results on the difference in accuracy, in each classifier, in the cases of with, or without, the snoring noise.

Metric	chi-square	p-value
Precision	31.5075	<0.0001
Recall	227.2857	<0.0001
F1	197.7716	<0.0001
Kappa	176.4255	<0.0001
Matthews	168.3359	<0.0001
AUC	287.8487	<0.0001

RQ4: To what extent does snoring impact defect prediction accuracy? - Discussion

- Snoring decreases accuracy in each of the 15 classifiers in each of the six accuracy metrics
- Difference is least significant in precision metric

RQ5: To what extent is no data better than snoring data? - Rationale

- Since snoring impacts prediction accuracy (RQ4), should we remove noise from the data
- Can't surgically remove noisy datapoints
- Only way to remove noise is to avoid potentially noisy data
- Will removing potentially noisy releases improve predictions

RQ5: To what extent is no data better than snoring data? - Design

- Dependent Variable: Same as RQ3
- Independent Variable : Number of releases to eliminate
- Remove the last 1,2,3,4 releases from the TrS dataset and test on TeNS
- When removing releases, keep defective classes
  - These classes can't possibly be noisy

RQ5: To what extent is no data better than snoring data? - Design



Ideal scenario

TrNs	TeNs	Future releases



Data collection time

TrS		Future releases



Data collection time

TrS-1			Future releases



Data collection time

TrS-2				Future releases

RQ5: To what extent is no data bi

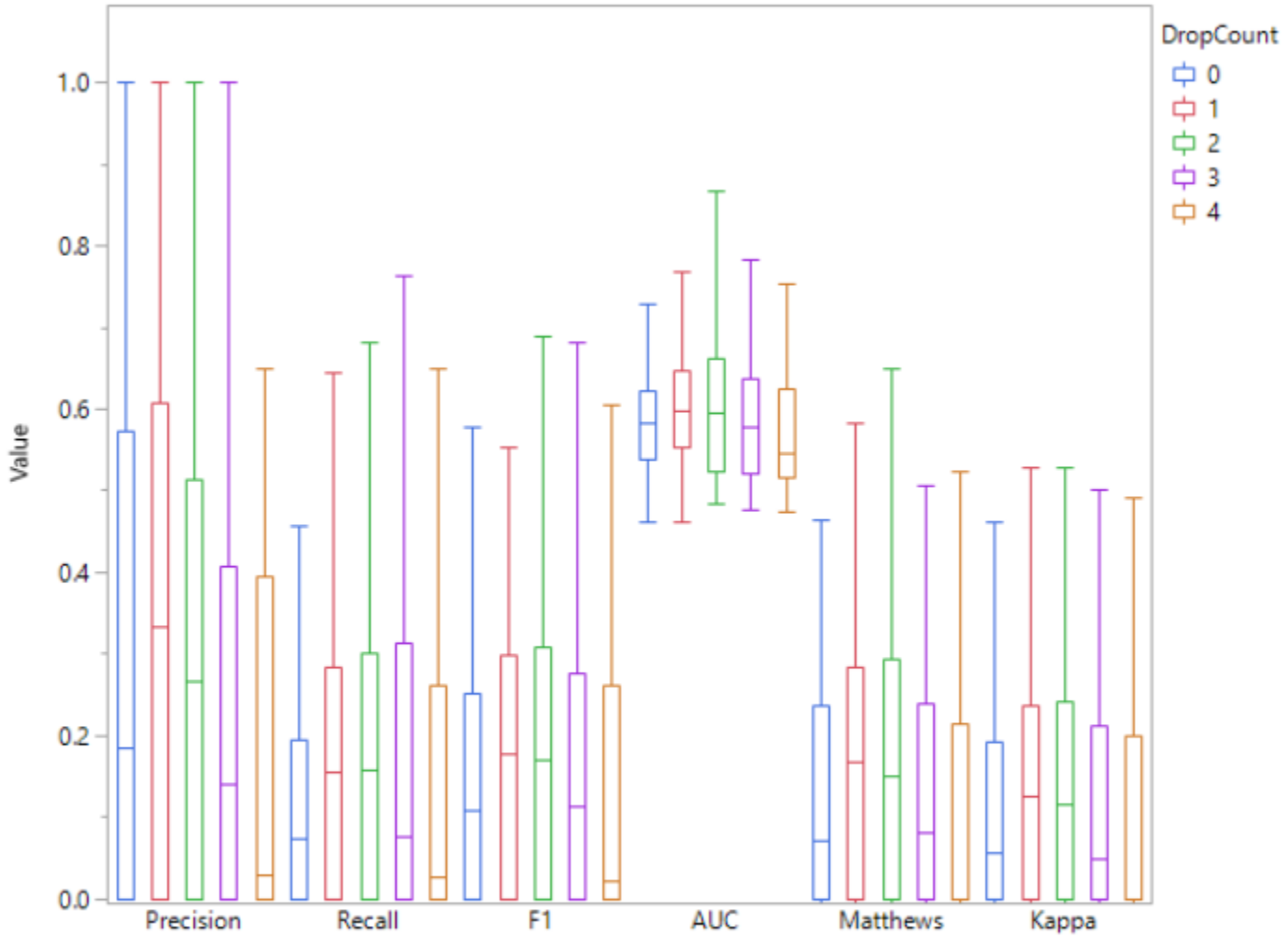


Figure 4.8: Distribution of accuracy, among datasets, in the cases of removal of the last releases of data.

RQ5: To what extent is no data better than snoring data? - Results

Table 4.8: Relative gain (%) in accuracy achieved by removing a specific number of releases of data, in average across classifiers and datasets.

Removed Releases	Precision	Recall	F1	Kappa	Matthews	AUC
1	0.17	0.41	0.33	0.39	0.32	0.03
2	-0.01	0.67	0.37	0.44	0.31	0.03
3	-0.27	0.45	0.07	-0.03	-0.12	0.02
4	-0.39	0.29	-0.09	-0.18	-0.25	0.00

RQ5: To what extent is no data better than snoring data? - Results

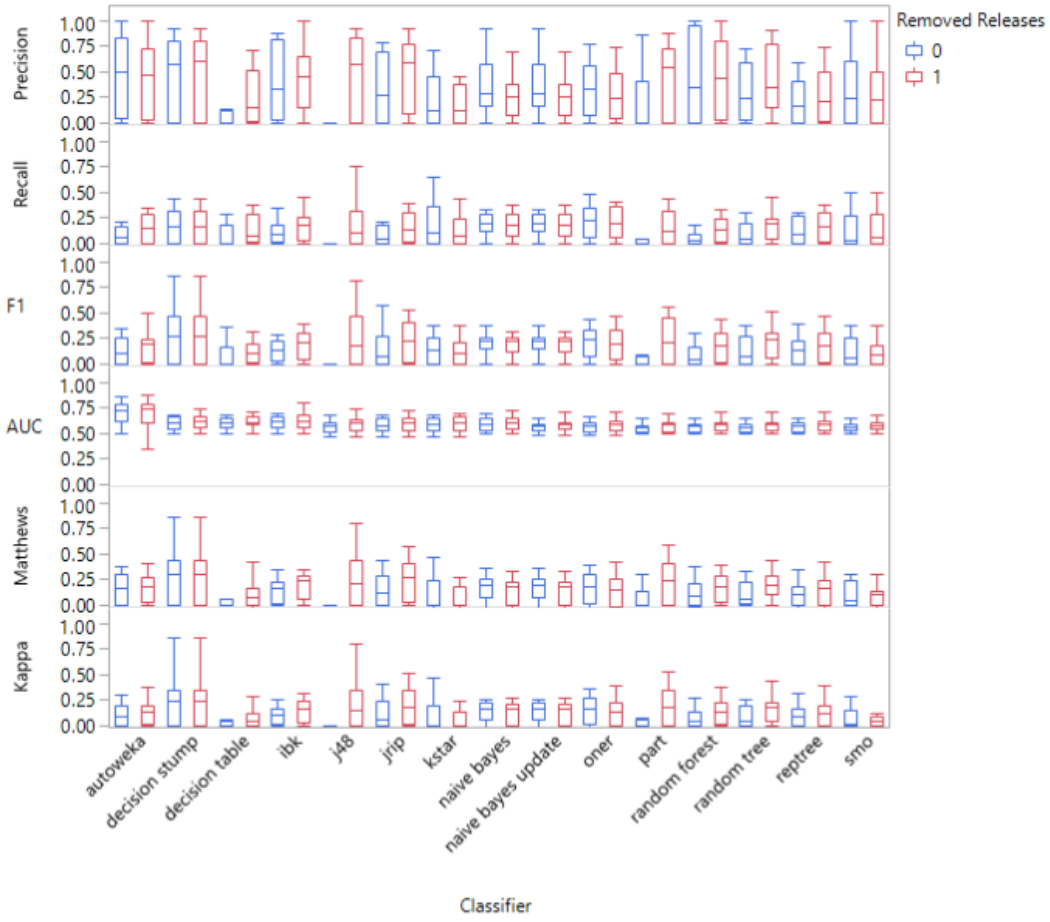


Figure 4.9: Accuracy in each classifier achieved by removing, or not, the last release data.

RQ5: To what extent is no data better than snoring data? - Results

Table 4.9: Statistical test results on the difference in accuracy, in the cases of removing, or not, the last release data.

Metric	chi-squre	p-value
Precision	3.4805	0.0621
Recall	9.0386	0.0026
F1	7.3716	0.0066
Kappa	5.6568	0.0174
Matthews	4.2771	0.0386
AUC	6.2638	0.0123

RQ5: To what extent is no data better than snoring data? - Discussions

- Removing one release is better than removing no release in all accuracy metrics
- Removing three or four releases is worse than removing one or two releases in all accuracy metrics
- Removing two releases is only better than removing one release in terms of Recall.
- The gain in removing releases is particularly small in terms of AUC
- Removing more than one release reduces precision

CONCLUSIONS-1

- Most of the defects in a project slept for more than 19% of the existing releases.
- The missing rate is more than 50% unless we remove more than 20% of the releases.
- The relative error in measuring the classifiers' accuracy achieved by using a dataset with snoring is about 100% in all accuracy metrics other than AUC.

Conclusions - 2

- The presence of snoring decreases the accuracy in each of the 15 classifiers, in each of the 6 accuracy metrics. For instance, Recall, F1, Kappa and Matthews decrease by about 80%.
- Removing one release of data is better than removing no data in all accuracy metrics. For instance, Recall, F1, Kappa and Matthews increases by about 30%

Future Works

- Fine grained noise removal
- Combine noise removal techniques
- Distribution as inputs
- Replication in the context of JIT